Phase 1 Proposal: Wikipedia Knowledge Integrity Lakehouse for Pakistan's Languages

Sep 27, 2026 · @Zuzi

Summary

Wikipedia isn't only in English. There are active Wikipedias in Urdu, Punjabi (Shahmukhi), Sindhi and Pashto as well. They're run by small groups of volunteers, and nobody really has a clear picture of how healthy they are. For our project we want to build that picture: a Spark pipeline that follows every edit, new page, deletion and revert on these four wikis, and a dashboard that shows whether they're growing, how fast vandalism gets cleaned up, where edit wars break out, and how much of the content is actually written by bots.

We picked this over a simpler dataset on purpose. The full history and the daily updates come from two different sources in two different formats, so we have to merge them into one model. The monthly history dump also rewrites the past (when a page is renamed, old records change too), which means we can't just keep appending. On top of that we need to handle deletions and renames properly, detect reverts across millions of revisions, and deal with real IP addresses in the older data.

Team	[Member 1 name, Roll No.] · [Member 2 name, Roll No.]
Course / Instructor	[Course name] · Amir Iqbal
GitHub repository	https://github.com/[your-username]/pk-wiki-integrity-lakehouse
Platform	Databricks Free Edition
Dashboard	Power BI Desktop
Data	Wikimedia history dumps (full load) and the MediaWiki recent changes API (incremental)

1. Domain & Source Identification

What the project is about. Our domain is knowledge integrity on Wikipedia, focused on four of Pakistan's languages: Urdu (`urwiki`), Western Punjabi (`pnbwiki`), Sindhi (`sdwiki`) and Pashto (`pswiki`). For many readers these wikis are the only free

encyclopedia in their own language, so it matters whether they're being looked after. The people who'd use our dashboard are the volunteer communities themselves, plus NGOs and groups working on digital literacy and regional languages.

Where the data comes from.

Used for	Source	Format
Full load	MediaWiki History dumps, one file per wiki: `https://dumps.wikimedia.org/other/mediawiki_history/<snapshot>/<wiki>/<snapshot>.<wiki>.all-time.tsv.bz2`	Tab-separated text, bzip2 compressed
Incremental load	MediaWiki Action API, recent changes list: `https://<lang>.wikipedia.org/w/api.php?action=query&list=recentchanges&rcstart=<watermark>&rcdir=newer`	JSON
Stretch goal for Phase 3	Wikimedia EventStreams live `recentchange` feed	Server-Sent Events (JSON)

How we'll load it. The history dump is our full load. It has every revision, page event and user event since 2001, and for small wikis like ours it comes as a single file, so one download per wiki gives us the whole baseline.

After that, a daily job asks the recent changes API for everything that happened since our last run (we save that timestamp as a watermark). This gives us new edits and pages, but also updates like page renames, and deletions, which show up as log events. We don't physically delete anything; we mark those pages with `is_deleted = true` so the history still adds up.

The tricky part is that the dump is rebuilt from scratch every month, because renames and moves change old records. Wikimedia actually warns that stitching dumps together incrementally gives inconsistent results. So once a month we'll swap in the newest snapshot for everything up to that month and keep only the daily API data after it. Getting this handover right is probably the hardest engineering problem in the project.

2. Data Samples & Volume

Sample files. We wrote a small Python script, `ingestion/ingest_wikimedia.py`, that produces both sample payloads. The samples are in `sample_data/raw/` in our repo (with PII masked first, see section 3).

For the full-load sample we run `python ingest_wikimedia.py full --snapshot 2026-08 --sample-lines 5000`. It streams each wiki's dump, decompresses it as it downloads, and keeps the first 5,000 rows per wiki, so we don't have to download gigabytes just to get a sample.

For the incremental sample we run `python ingest_wikimedia.py incremental`, which pulls the last 24 hours of changes for each wiki and saves them as JSON Lines. Every record is wrapped with a few extra fields (which wiki, which URL, when we fetched it, and the time window) so we can always trace a row in Bronze back to where it came from. Running it again the next day picks up exactly where the first run stopped.

What the samples actually look like. We ran both loads on 27 September 2026 (dump snapshot 2026-08):

Sample	Rows	Size	Notes
Full load, 4 wikis × 5,000 rows	20,000 (17,106 revisions, 2,671 page events, 223 user events)	7.8 MB (1.85 – 2.18 MB per wiki)	78 columns per row; rows start in 2002–2004, the earliest history of each wiki
Incremental, 24 hours, 4 wikis	17,405 change events	11.6 MB	Urdu 17,045 events, Sindhi 350, Punjabi 6, Pashto 4

The incremental sample surprised us. Urdu Wikipedia alone had over 17,000 changes in one day, and about half of all events (8,782) were made by bots. Most of the rest were category updates and normal edits, plus 31 log events: 16 page moves, 4 user renames, 3 new accounts, 1 page deletion and 1 protection. So the daily feed is bigger than we first assumed, and it already includes updates and deletes for us to handle.

Volume estimate for the real pipeline. A history row is about 400 bytes uncompressed, and the daily feed is about 11–12 MB for roughly 17,000 events.

Load	Rows (approx.)	Size (approx.)	How often
Initial full load, 4 wikis	5 – 10 million events	2 – 4 GB uncompressed, around 0.5 – 1 GB as .bz2	Once, then refreshed monthly
Daily incremental	15,000 – 20,000 events	10 – 15 MB JSON	Every day at 2 AM PKT
Weekly incremental	100,000 – 140,000 events	70 – 100 MB JSON	Seven daily runs

Load	Rows (approx.)	Size (approx.)	How often
Stored as Delta tables	same rows, about 30 of 78 columns kept	0.3 – 0.8 GB	—

That's big enough that Spark genuinely helps: it reads bzip2 files in parallel chunks without unzipping them first. If the full size is too heavy for the free tier, our fallback is Urdu Wikipedia only, from 2015 onward. We can also drop the `categorize` events, which are almost half the feed and matter least for our questions.

3. Security & Compliance

Does the data have PII? Yes, and some of it is quite sensitive. Until 2025, anyone who edited Wikipedia without logging in was shown publicly by their IP address. Those old IP addresses are still in the historical data. Wikimedia has since switched to temporary accounts, so newer logged-out edits show an auto-generated name like `~2026-12345-6` instead of an IP. That means our pipeline has to deal with both: exact IPs in older records and pseudonymous names in newer ones.

These are the fields that carry personal data, and what we plan to do with each one before it reaches Silver:

Field	What it contains	What we'll do
Editor name for logged-out edits (older data)	Exact IPv4 or IPv6 address	Look up the country using the free DB-IP Lite database, keep a salted hash so we can still count unique editors, then drop the IP completely
Editor name for registered users	Username, which is often someone's real name	Replace with a salted SHA-256 hash (`editor_key`). Bot accounts are public service accounts, so we keep those names
Temporary account names	Pseudonymous name tied to a browser cookie	Hash it the same way and mark the editor type as temporary
Edit summaries	Free text that sometimes includes names, emails or IPs	Mask anything that looks like an IP, email or user mention with regex, keep the rest

Field	What it contains	What we'll do
Block log targets	The IP range or username that got blocked	Hash the target; keep only how long the block was and the reason type

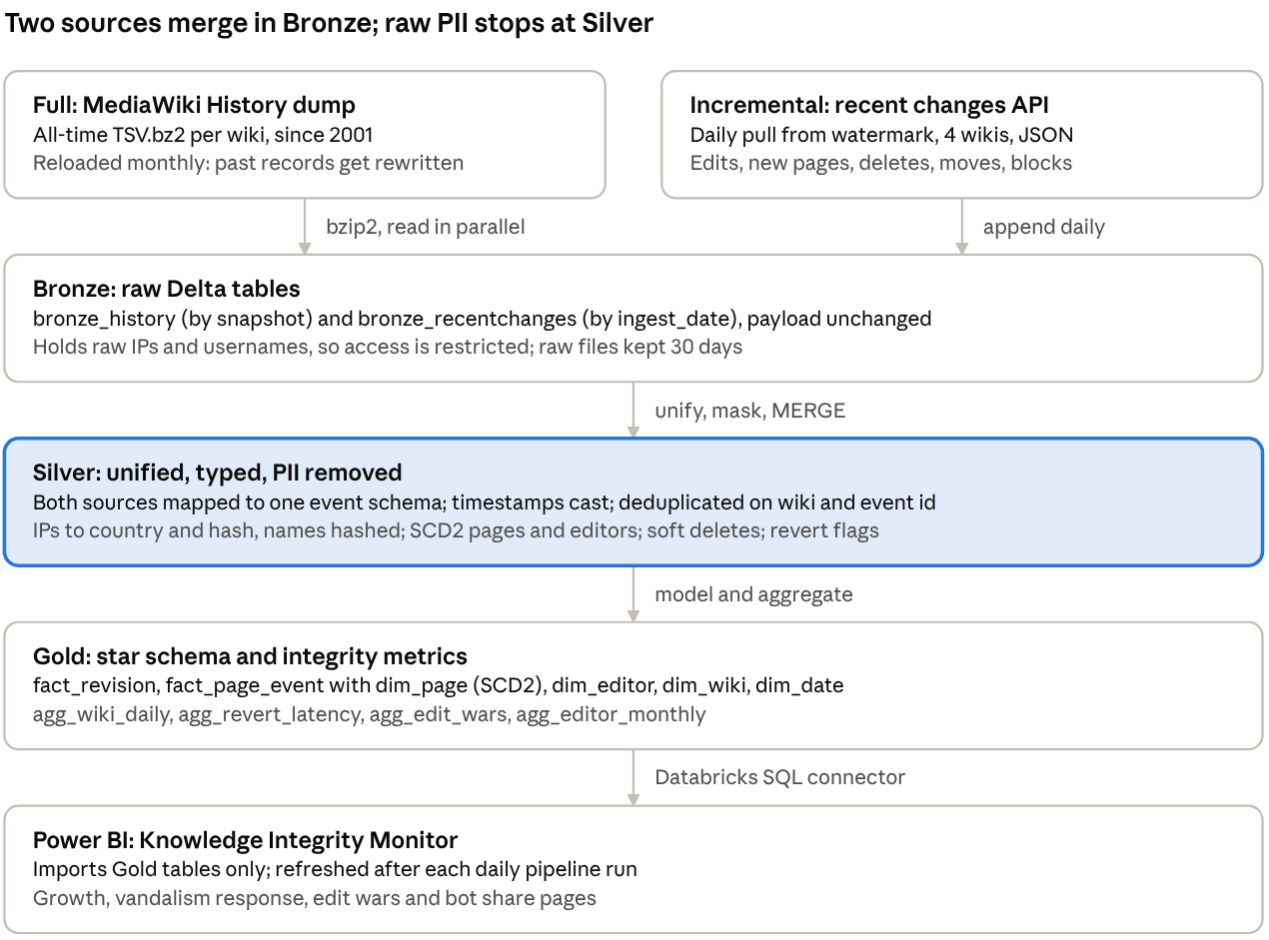
Our samples confirm this is a real issue, not a hypothetical one. In the 20,000 history rows, 874 edits (4.4%) show a raw IP address as the editor, and some rows show registered users' real full names. In the 24-hour incremental sample there were no IPs at all, but 348 events came from temporary accounts, which matches the switch Wikimedia made in 2025. We've already written the masking step as `common/pii_policy.py` and run it on the samples; the files in our repo contain only hashed editor IDs (bot names are kept).

How we'll keep it safe. Raw data stays in Bronze, and only the two of us have access to that schema. Raw files are deleted 30 days after they've been processed. All the hashing and masking lives in one shared function, `apply_pii_policy()`, which runs in the Bronze to Silver step, so no raw IP or username ever reaches Silver, Gold or the dashboard.

The hash salt is stored as a Databricks secret, not in our code or on GitHub. The sample files we push to our public repo go through the same masking first. We also want to be good API citizens, so our script identifies itself with a proper User-Agent, sends requests one at a time, and never downloads article text.

4. High-Level Medallion Data Modeling

The overall flow is simple to describe: two sources land in Bronze, get cleaned and merged into one privacy-safe model in Silver, and end up as a star schema with ready-made metrics in Gold.



Medallion pipeline · source to dashboard

Each arrow is one notebook in a single scheduled Databricks Job. Bronze only ever appends, while Silver and Gold use `MERGE` or `replaceWhere`, so if a run fails halfway we can just run it again without creating duplicates.

Getting from Bronze to Silver. Most of the real work happens here:

1. The dump files have no header row, so we'll define the schema ourselves (78 columns, which we mapped by profiling our samples against Wikimedia's documentation). The API JSON gets exploded and flattened.
2. Both sources are mapped onto one common event table, so an edit looks the same whether it came from the dump or the API.
3. Timestamps come in two formats (`YYYY-MM-DD HH:MM:SS.S` in the dump, ISO-8601 from the API). We convert both to UTC timestamps and add a Pakistan-time column for the dashboard.
4. We remove duplicates using the wiki plus the revision or log ID. Daily windows overlap a little, and where the dump and the API cover the same period, the dump wins.
5. When a new monthly dump arrives, we replace everything up to that month and keep the API data after it.

6. Reverts: the dump already tells us which edits were reverted and how quickly. For API data we work it out ourselves. If an edit has the same content hash (`sha1`) as an earlier version of that page, it has undone everything in between. We'll do this with window functions over each page's history.
7. Page renames and editor changes (a rename, a new bot flag, a temporary account becoming registered) are tracked as SCD Type 2, so we keep the old version instead of overwriting it.
8. Finally the PII rules from section 3 run, followed by basic quality checks. Rows that fail go to a quarantine table instead of breaking the job.

Silver tables we're planning.

Table	One row per	Main columns
silver_revision	revision	wiki, rev_id, page_id, rev_ts, editor_key, editor_type (registered, temporary, IP, bot), ip_country, bytes, bytes_diff, sha1, is_minor, comment_masked, is_revert, is_reverted, seconds_to_revert, is_deleted, source
silver_page_event	page log event (create, move, delete, restore, protect)	wiki, log_id, page_id, event_type, old_title, new_title, event_ts, editor_key
silver_page	version of a page (SCD2)	wiki, page_id, title, namespace, is_redirect, valid_from, valid_to, is_current, is_deleted
silver_editor	version of an editor (SCD2)	editor_key, editor_type, is_bot, registration_ts, valid_from, valid_to, is_current
silver_block_event	block	wiki, target_key, block_ts, duration_hours, reason_category

Gold layer. For Gold we'll build a star schema that Power BI can use directly, plus a few pre-computed tables for the heavier calculations so the dashboard stays fast.

Table	Type	What's in it
fact_revision	Fact	One row per edit: size change, whether it was reverted, how long that took, and a vandalism-suspect flag (reverted within 24 hours by someone else)
fact_page_event	Fact	Page creations, renames, deletions and restores
dim_wiki, dim_date	Dimensions	Language and script of each wiki; a calendar that flags Pakistani public holidays
dim_page, dim_editor	SCD2 dimensions	Page title history; editor type, bot flag, first edit date, new vs returning
agg_wiki_daily	Aggregate	Edits, new articles, deleted pages, active editors and bot share per wiki per day
agg_revert_latency	Aggregate	Median and 90th-percentile time to revert, by wiki, month and editor type
agg_edit_wars	Aggregate	Pages with at least 3 reverts by 2 or more editors within 24 hours
agg_editor_monthly	Aggregate	Active editors (5+ edits), new editors, and how many came back the next month

5. Business Intelligence & Dashboard

We'll build the dashboard in Power BI and call it the Knowledge Integrity Monitor. It will have a filter bar at the top for wiki, date range, and whether to include bots. These are the questions we want it to answer:

1. Are the Urdu, Punjabi, Sindhi and Pashto Wikipedias actually growing, both in new articles and in active editors?
2. When someone vandalises a page, how long does it take for the community to undo it? Is that different for registered, temporary and IP editors?
3. Which pages are the most fought over, and what are they about?
4. How much of each wiki is written by bots rather than people?
5. Did anonymous editing or vandalism change after Wikipedia replaced IP editing with temporary accounts?

The last question is the one we're most curious about. The rollout is a real-world change with a clear before and after, and nobody has looked at it for these wikis yet as far as we know.

Visuals we're planning.

Visual	Chart	Answers
Growth over time	Line chart of new articles and active editors per month, one line per wiki	Q1
How fast vandalism is fixed	Box plot of time-to-revert by editor type, with a card showing the median in minutes	Q2
Edit-war leaderboard	Table of the 20 most contested pages, with revert count and number of editors involved	Q3
Humans vs bots	100% stacked area chart of bot vs human edits per month	Q4
Before and after temporary accounts	Column chart comparing anonymous edit share and revert rate in the 6 months before and after each wiki's rollout	Q5

If we have time, we'd also like to add a heat map of edits by hour and day of week (in Pakistan time) to show when each community is most active.

6. Engineering Setup & FinOps

GitHub. Our repository is https://github.com/[your-username]/pk-wiki-integrity-lakehouse. We each work on our own branch and merge through pull requests, so we review each other's code before it goes in. The layout we've set up:

```
pk-wiki-integrity-lakehouse/
├── README.md                          # overview, setup, architecture diagram
├── docs/phase1_proposal.pdf
├── ingestion/ingest_wikimedia.py      # full dump + incremental API extractor
├── notebooks/
│   ├── 01_bronze_ingest.py
│   ├── 02_silver_unify_and_mask.py
│   ├── 03_silver_scd2_and_reverts.py
│   └── 04_gold_model.py
├── common/pii_policy.py               # shared apply_pii_policy()
├── sample_data/raw/
│   ├── mediawiki_history/load_type=full/...        # full-load sample
│   └── recentchanges/load_type=incremental/...     # incremental sample
├── dashboard/                         # .pbix (Phase 3)
└── .gitignore                         # .env, secrets, full dumps
```

Infrastructure. We'll use Databricks Free Edition for Spark, Delta Lake and scheduled jobs, and Power BI Desktop (also free) for the dashboard, connected through the Databricks SQL connector. Both data sources are free and don't need an API key, so the whole project costs nothing to run.

Not burning through the free tier. Free compute is limited, so we're planning around it from the start:

- We'll build and test every notebook on the small samples and on Urdu Wikipedia only. The full four-wiki load only runs once everything works on the samples.
- The dumps stay compressed as `.bz2` files, which are several times smaller than the raw text, and Spark reads them directly.
- We drop what we don't need early: about 48 of the 78 dump columns, and page types we aren't analysing (like talk pages).
- After the first load, daily runs only handle a few MB, and the monthly refresh only rewrites the partitions that changed.
- Jobs run on serverless compute that shuts down when the run finishes, so nothing sits idle eating credits.
- Once a week we run `OPTIMIZE` and `VACUUM`, raw Bronze files are deleted after 30 days, and old dump snapshots are removed once the new one is in. We're aiming to stay under 3 GB of storage in total.
- GitHub only holds our code and small masked samples. The full dumps are excluded with `.gitignore`.